Status	Started	Completed
Finished	Wednesday, 11 November 2026, 1:31 PM	Wednesday, 11 November 2026, 1:41 PM

Duration	Grade
10 mins	**56.00** out of 60.00 (**93.33**%)

Question 1

Flag question

Correct Mark 1.00 out of 1.00

Your agent handles complex multi-step deployments. Step 6 is a database schema migration that cannot be rolled back. The agent has high confidence the migration script is correct. What should the design mandate at step 6?

a. Proceed — high confidence justifies autonomous execution of irreversible steps.
b. Trigger a mandatory human-in-the-loop checkpoint before executing any irreversible action, regardless of the agent's confidence level. ✓
c. Require the agent to log the migration script before executing it.
d. Have the agent execute the migration on a staging environment first, then production.

Question 2

Flag question

Correct Mark 1.00 out of 1.00

Your coordinator agent passes its full 40-turn conversation history to each subagent it spawns. Subagents frequently produce outputs that reference prior topics unrelated to their assigned task. What is the architectural fix?

a. Instruct subagents via the system prompt to ignore unrelated topics from the history they receive.
b. Pass only the context explicitly relevant to each subagent's specific task — subagents should receive only what they need, not the coordinator's full history. ✓
c. Use a shared memory store so all subagents read only what they need from a central location.
d. Increase the subagents' context windows so they can process the full coordinator history without confusion.

Question 3

Flag question

Correct Mark 1.00 out of 1.00

Your agentic loop correctly checks stop_reason but continues running after receiving 'end_turn'. Investigation shows the loop re-enters after the 'end_turn' check passes. Where is the most likely bug?

a. The Claude API is intermittently returning incorrect stop_reason values.
b. The model requires an additional 'confirm completion' tool call before accepting end_turn.
c. The loop appends the assistant response to history and calls the API again before the stop_reason check is evaluated — the stop_reason check must come immediately after receiving the response and before any further processing or API calls. ✓
d. The max_tokens setting is too low, causing early end_turn signals.

Question 4

Flag question

Correct Mark 1.00 out of 1.00

Your agent is executing a 12-step database migration workflow. At step 9, it receives a permission error trying to modify a production table. After three retries, it proceeds to step 10 despite step 9 still being incomplete. What architectural change is required?

a. Log the step 9 failure and continue — partial migrations can be completed later.
b. Treat step 9's permission error as a non-retryable failure after reading isRetryable: false from the structured error. Stop the workflow at step 9, escalate with structured context about what failed and what access is needed, rather than proceeding with an incomplete migration. ✓
c. Have step 10 detect and compensate for the step 9 incompletion automatically.
d. Add more retries — permission errors sometimes resolve after multiple attempts.

Question 5

Flag question

Correct Mark 1.00 out of 1.00

Your coordinator receives: {isError: true, errorCategory: 'permission', isRetryable: false, message: 'Access denied to restricted financial report'}. The coordinator retries the request three times. What does this indicate about the coordinator's error handling?

a. The permission error may become retryable after a delay — four retries is reasonable.
b. The coordinator is ignoring the isRetryable: false flag — permission errors are permanent and retrying wastes resources. The coordinator should read the errorCategory and isRetryable fields, skip retries for non-retryable permission errors, and annotate the report section as unavailable. ✓
c. The coordinator should escalate to a human after three retries.
d. The coordinator is correctly being persistent about recovering the data.

Question 6

Flag question

Correct Mark 1.00 out of 1.00

Your agent is deployed in an environment where a subagent occasionally fails mid-task. You need the coordinator to continue with partial results rather than terminating the whole workflow. What error handling pattern should subagents implement?

a. Subagents should silently swallow errors and return empty results to avoid interrupting the coordinator.
b. The coordinator should retry the entire subagent task from scratch on any error.
c. Subagents should raise exceptions that terminate the entire coordinator workflow immediately.
d. Subagents should implement local recovery for transient failures and, for unrecoverable errors, return structured error context including failure type, what was attempted, and any partial results — enabling the coordinator to make an informed decision about how to proceed. ✓

Question 7

Flag question

Correct Mark 1.00 out of 1.00

Your agent often autonomously handles policy exception cases that always end in failed resolutions and unhappy customers. The system prompt says 'always try to resolve before escalating unless absolutely necessary.' What is the problem?

a. Add a confidence threshold: escalate when confidence falls below 0.4.
b. Remove the resolution-first instruction so the agent escalates by default.
c. The agent needs more training examples of policy exception handling.
d. 'Unless absolutely necessary' provides no actionable criteria. Replace it with explicit, enumerated conditions that trigger immediate escalation — specific case types, amount thresholds, and account flags that the agent should not attempt to resolve autonomously. ✓

Question 8

Flag question

Incorrect Mark 0.00 out of 1.00

You need to execute three independent research subtasks simultaneously to minimise latency. Using the Agent SDK and Task tool, how do you trigger all three in parallel?

a. Emit one Task call per coordinator turn across three separate turns, then aggregate.
b. Configure allowedTools with parallel: true on each subagent's AgentDefinition.
c. Use asyncio.gather() in the application layer to run three separate coordinator sessions simultaneously. ✗
d. Emit all three Task tool calls in a single coordinator response — the SDK executes multiple Task calls from one response concurrently.

Question 9

Flag question

Correct Mark 1.00 out of 1.00

Your coordinator agent uses a fixed pipeline: always invoke web_search, then document_analysis, then synthesis, regardless of the query. A user asks a simple factual question answerable from a single document. What is the problem with this approach?

a. The coordinator should run all subagents in parallel rather than sequentially to reduce latency.
b. The fixed pipeline over-provisions subagents — for simple queries, a model-driven approach that dynamically selects only the needed subagents reduces latency and cost. ✓
c. Fixed sequential pipelines are more reliable because they produce predictable behavior.
d. Simple queries should be handled by a smaller, cheaper model rather than the full pipeline.

Question 10

Flag question

Correct Mark 1.00 out of 1.00

The web search subagent times out. What is the best error propagation approach?

a. Propagate the timeout to the top-level coordinator, terminating the entire workflow.
b. Auto-retry with exponential backoff, return 'search unavailable' after retries are exhausted.
c. Catch the timeout, return an empty result marked as successful.
d. Return structured error context (failure type, attempted query, partial results, alternatives). ✓

Question 11

Flag question

Correct Mark 1.00 out of 1.00

Your coordinator always routes queries through all four subagents regardless of complexity. A user asks 'What is the current date?' The coordinator invokes web search, document analysis, data validation, and synthesis before responding. What design principle is being violated?

a. All queries must go through the same pipeline for consistency and auditability.
b. Subagents should self-select based on the query rather than being chosen by the coordinator.
c. Coordinators should never answer directly — all responses must go through subagents.
d. The coordinator should use dynamic subagent selection: analyse the query requirements and invoke only the subagents necessary. A trivial factual question requires no subagent delegation. ✓

Question 12

Flag question

Incorrect Mark 0.00 out of 1.00

A report on 'impact of AI on creative industries' only covers visual arts. The coordinator decomposed the topic into only visual arts subtasks. All subagents completed successfully. What is the root cause?

a. The synthesis agent lacks gap-identification instructions. ✗
b. The coordinator's task decomposition is too narrow.
c. Web search queries were not comprehensive enough.
d. Document analysis filtered out non-visual sources.

Question 13

Flag question

Correct Mark 1.00 out of 1.00

Your multi-agent research system produces a report where the synthesis agent has combined findings from a 2024 academic paper (claims X causes Y) with a 2022 industry report (claims Y causes X) and presented both as fact without noting the contradiction. What is missing from the pipeline design?

a. Subagents should return structured outputs including source metadata and confidence; the synthesis agent should be explicitly instructed to annotate conflicting findings with their source attribution, publication dates, and a conflict_detected flag rather than silently merging contradictory claims.
b. A post-synthesis fact-checker that searches for contradictions in the final report.
c. A spell-checker to catch factual errors.
d. Better few-shot examples for the synthesis agent.

Question 14

Flag question

Correct Mark 1.00 out of 1.00

Two subagents return conflicting statistics: web search reports 34% growth, document analysis reports 41% growth for the same period. Both sources appear credible. How should the coordinator handle this?

a. Discard both and have web search retry with different queries until a consensus figure emerges.
b. Prefer the document analysis figure — primary source documents are inherently more reliable.
c. Retain both values with full source metadata and pass both to synthesis with explicit conflict annotation for reasoned resolution.
d. Average the two values and pass 37.5% to synthesis as a reconciled figure.

Question 15

Flag question

Correct Mark 1.00 out of 1.00

Your multi-agent system needs to handle a case where the synthesis subagent discovers a critical gap in the research after partial synthesis. It has already incorporated findings from two of three subagents. What should it do?

a. Return a structured partial result to the coordinator with explicit gap annotation — indicating which topics are well-covered, which are missing, and what additional research would be needed — so the coordinator can decide whether to re-delegate or proceed.
b. Fabricate plausible content to fill the gap so the report appears complete.
c. Terminate and restart the entire research workflow from the beginning.
d. Complete the synthesis with available data and note the gap in the report.

Question 16

Flag question

Correct Mark 1.00 out of 1.00

A customer support agent receives a request that mentions both a billing dispute and a shipping problem. The agent handles only the billing dispute and marks the case closed. What task decomposition pattern prevents this?

a. Limit agents to one issue per session to avoid multi-concern confusion.
b. Decompose multi-concern requests into distinct items, investigate each in parallel using shared context, then synthesize a unified resolution before closing the case. ✓
c. Ask the customer to submit each issue as a separate support ticket.
d. Route multi-issue requests immediately to a human agent to avoid partial resolution.

Question 17

Flag question

Correct Mark 1.00 out of 1.00

Your MCP tool must guarantee the model calls it (rather than returning a text response) when the user asks any question about customer orders. Which tool_choice setting achieves this guarantee?

a. tool_choice: {type: 'any'} — this forces the model to call some tool on every turn, ensuring structured output when multiple tools exist.
b. tool_choice: auto — the model will naturally select the tool when relevant.
c. tool_choice: {type: 'tool', name: 'lookup_order'} — this forces the model to call the specific order lookup tool, guaranteeing it runs for every order query. ✓
d. Add 'always use this tool' to the tool description.

Question 18

Flag question

Correct Mark 1.00 out of 1.00

Your MCP server connects to three different databases — customer records, order history, and product catalog — each with different response time characteristics. Customer records respond in <100ms, order history in 1–3 seconds, product catalog in 5–8 seconds. How should you design the tool interfaces for latency management?

a. Add a 10-second timeout to all three tools and treat timeouts as empty results.
b. Use a single unified 'get_all_data' tool that fetches from all three databases in one call.
c. Design customer records as a synchronous tool (fast enough to block); design order history and product catalog as async tools with polling tokens so the agent can request both and then check their status while doing other work. ✓
d. Make all three tools synchronous since the agent must wait for all data anyway.

Question 19

Flag question

Correct Mark 1.00 out of 1.00

The process_refund tool returns {"status": "error", "message": "Operation failed"} on database connection failures. The agent enters an infinite retry loop. What is missing from this error response?

a. The error should be thrown as a Python exception so the agent's try/except handler can catch it.
b. The response should return an empty result with status 'success' so the agent continues processing.
c. The message should include the database connection string to aid debugging.
d. The response lacks errorCategory and isRetryable fields — without these the agent cannot distinguish a transient connection timeout from a permanent failure and defaults to blind retrying. ✓

Question 20

Flag question

Incorrect Mark 0.00 out of 1.00

Your .mcp.json configuration file will be committed to the shared team repository. The server requires an API key. How should the key be stored?

a. Create a secrets.json file that is .gitignored and distribute it manually to each developer. ✗
b. Use environment variable expansion: {"apiKey": "${INDEXER_API_KEY}"} with each developer setting the actual value in their local shell environment.
c. Store the key in CLAUDE.md under a [secrets] section.
d. Store the key directly as a string value: {"apiKey": "sk-prod-abc123"}.

Question 21

Flag question

Incorrect Mark 0.00 out of 1.00

Your agent has access to 18 tools across file operations, database queries, git, static analysis, and internal APIs. It frequently selects the wrong tool. What is the most effective architectural fix?

a. Reduce the agent's active tool set to 4–5 core tools and distribute the rest across specialized subagents with scoped access.
b. Deploy a pre-processing NLP classifier that pre-selects tools before the agent runs.
c. Rewrite all 18 tool descriptions to be longer and more detailed.
d. Add 12 few-shot examples in the system prompt demonstrating correct tool selection for common requests. ✗

Question 22

Flag question

Correct Mark 1.00 out of 1.00

Your document analysis tool returns [] for password-protected files it cannot decrypt. The agent reports these as having no findings, passing them through quality checks as clean. How should the tool response differentiate these cases?

a. Exclude password-protected files from the analysis scope entirely.
b. Return null instead of [] for files that cannot be accessed.
c. Throw an exception for password-protected files.
d. Return an explicit status indicating the reason: {status: 'inaccessible', reason: 'password_protected', results: []} versus {status: 'complete', results: []} for a genuinely clean file — the agent needs this distinction to route inaccessible files appropriately. ✓

Question 23

Flag question

Correct Mark 1.00 out of 1.00

Your order history tool returns 45 fields per order. A session with 15 order lookups accumulates 675 field values in context. The agent starts missing task-critical details. What is the tool design fix?

a. Increase max_tokens to accommodate the verbose order history.
b. Add a summary tool that condenses order data after each lookup.
c. Cache all tool responses externally and retrieve them on demand.
d. Redesign the tool to return only the 5–6 fields relevant to common use cases by default, with an option to request additional fields for specific orders — verbose responses accumulate and crowd out important information. ✓

Question 24

Flag question

Correct Mark 1.00 out of 1.00

Your agent has two similar MCP tools — analyze_content and analyze_document — with near-identical descriptions. The agent frequently routes document analysis requests to analyze_content. What is the correct fix?

a. Rename and rewrite both descriptions: give each a distinct purpose, accepted input format, output contract, and explicit guidance on when to use it over the other. ✓
b. Add a PreToolUse hook that validates selection and redirects incorrect calls.
c. Merge both tools into a single analyze_any tool to eliminate the ambiguity.
d. Add few-shot examples showing when to use each tool.

Question 25

Flag question

Correct Mark 1.00 out of 1.00

Your MCP server uses ${ANALYTICS_KEY} in .mcp.json. After a team expansion, three new developers report the MCP server fails to connect. What is the most likely cause and fix?

a. The new developers have not set ANALYTICS_KEY in their local shell environment. Create a .env.example file documenting all required environment variables so onboarding developers know exactly which values to set.
b. The .mcp.json syntax is wrong — fix the environment variable reference format.
c. The ${} expansion only works on Unix — Windows developers need a different config.
d. The MCP server is overloaded by the additional developers.

Question 26

Flag question

Correct Mark 1.00 out of 1.00

Your financial reporting tool returns 60 fields per report. After 8 report lookups in a session, the agent begins missing task-critical values stated earlier in the conversation. What is the most direct tool design fix?

a. Return only the 6–8 fields most relevant to common reporting tasks by default, with a separate detail endpoint for full field access when needed — verbose responses accumulate and crowd out important conversational context.
b. Add a report summary endpoint that the agent can call after each full report.
c. Cache all report data externally and retrieve specific fields on demand.
d. Increase max_tokens to handle the accumulated report data.

Question 27

Flag question

Correct Mark 1.00 out of 1.00

Three MCP tools in your pipeline return timestamps in Unix epoch, ISO 8601, and 'Mon Jan 06 2025' format respectively. The model inconsistently converts between formats, causing calculation errors. What is the cleanest fix?

a. Implement a PostToolUse hook that normalises all three timestamp formats to ISO 8601 before any tool result enters the conversation history — deterministic transformation every time regardless of which tool ran.
b. Prompt the model: 'Convert all timestamps to ISO 8601 before using them in calculations.'
c. Ask all three tool providers to update their response format.
d. Add timestamp format conversion instructions to each tool's description.

Question 28

Flag question

Correct Mark 1.00 out of 1.00

Your CI pipeline's Claude Code review produces a long prose report that downstream tools cannot parse reliably. You need structured findings with specific fields. Which CLI approach enforces this?

a. Use the --structured flag in the claude CLI invocation.
b. Use claude -p with --output-format json and --json-schema pointing to a findings schema — this combination enforces structured, schema-compliant JSON output that CI tooling can parse reliably. ✓
c. Redirect stdout to a parser script that converts prose to structured data.
d. Add 'format your output as JSON' to the review prompt.

Question 29

Flag question

Correct Mark 1.00 out of 1.00

You are comparing three refactoring strategies for a legacy authentication system. All three start from the same initial codebase analysis. You want each exploration to be independent and not pollute each other or the main session. What is the correct approach?

a. Use fork_session three times from the shared analysis baseline — each strategy gets its own independent branch that does not affect the others or the main session. ✓
b. Run all three strategies sequentially in the main session, reverting changes between each.
c. Create three separate repositories to isolate the explorations.
d. Use /compact between each strategy to reset the context.

Question 30

Flag question

Correct Mark 1.00 out of 1.00

You need to compare two refactoring approaches from a shared codebase analysis baseline without affecting the main session. Which session management feature enables this?

a. /compact to create a summary that can be used as the starting point for each approach.
b. fork_session to create two independent branches from the shared analysis baseline, each exploring a different approach. ✓
c. --resume with two different session names pointing to the same checkpoint.
d. Start two completely fresh sessions, both seeded with a summary of the shared analysis.

Question 31

Flag question

Correct Mark 1.00 out of 1.00

Your team maintains a shared security_standards.md file updated by the security team monthly. You want CLAUDE.md to always use the current version without manual updates to CLAUDE.md each month. How do you achieve this?

a. Copy the security standards content into CLAUDE.md at the start of each month.
b. Store the URL of the standards document in CLAUDE.md and fetch it at session start.
c. Add @./security/security_standards.md to CLAUDE.md — the @import syntax loads the external file at session start, so updates to the referenced file automatically take effect without modifying CLAUDE.md. ✓
d. Create a PostToolUse hook that injects the security standards after each file edit.

Question 32

Flag question

Correct Mark 1.00 out of 1.00

Your code review agent flags issues but developers do not know how severe each one is. You want the output to include a severity field with values: critical, high, medium, low. Developers are getting inconsistent severity assignments for similar issues. What prompt engineering technique fixes this?

a. Provide a 4-level severity rubric in the prompt with a concrete code example for each level — showing exactly what a 'critical' issue looks like versus 'low' — so the model can apply consistent criteria across varied code patterns. ✓
b. Add a general instruction: 'Assign severity based on the impact of the issue.'
c. Add confidence scoring and filter out low-confidence severity assignments.
d. Use a temperature of 0 to eliminate variability in severity assignments.

Question 33

Flag question

Correct Mark 1.00 out of 1.00

Your /analyse-performance skill sometimes runs without the developer specifying which module to analyse and produces generic output. How do you force the developer to provide the module path before execution?

a. Add a system prompt instruction: 'Ask for the module path if not provided.'
b. Set argument-hint: 'Module path to analyse (e.g. src/api/auth.py)' in the SKILL.md frontmatter — Claude Code prompts the developer for the required argument when the skill is invoked without one. ✓
c. Add a default module path to the skill prompt.
d. Require developers to always provide arguments using the --args flag.

Question 34

Flag question

Correct Mark 1.00 out of 1.00

You need path-specific rules that apply only to Terraform files (*.tf) regardless of which directory they are in. Your project has Terraform files in infra/, modules/, and environments/ directories. Which configuration approach handles all locations correctly?

a. Create a CLAUDE.md in each of infra/, modules/, and environments/ with the Terraform rules.
b. Create a .claude/commands/terraform-review.md slash command that loads the Terraform rules when invoked.
c. Add Terraform rules to the root CLAUDE.md under a '## Terraform' header.
d. Create a .claude/rules/terraform.md file with YAML frontmatter: paths: ["**/*.tf"] — the glob pattern activates these rules for any .tf file regardless of directory location. ✓

Question 35

Flag question

Correct Mark 1.00 out of 1.00

A developer asks: 'Should I add a date validation check to the create_order function?' The function is well-understood, the fix is a single conditional, and there is a clear stack trace pointing to the exact location. Which mode is appropriate?

a. Direct execution — this is a well-scoped single-file change with a clear fix; plan mode would add overhead without benefit. ✓
b. Fork the session before making changes to preserve the prior analysis.
c. Use the Explore subagent to first map all usages of create_order before making the change.
d. Plan Mode — any code change should be planned before execution.

Question 36

Flag question

Correct Mark 1.00 out of 1.00

You are implementing a complex library migration that will touch 45+ files and requires choosing between two different migration approaches before writing any code. Which Claude Code workflow is most appropriate?

a. Use a CI/CD pipeline to run trial migrations on a branch and measure the results.
b. Direct execution — start migrating immediately and adjust as issues are discovered.
c. Plan Mode — explore the codebase, understand dependencies, evaluate both approaches and their tradeoffs, then present the migration plan for your approval before any files are touched. ✓
d. Ask Claude to write all migration code in a single large prompt.

Question 37

Flag question

Correct Mark 1.00 out of 1.00

Files in your codebase were heavily modified overnight. You have a --resume session from yesterday's investigation. What is the recommended approach for today's session?

a. Start a fresh session with a structured summary of yesterday's key findings — when the files under investigation have changed substantially, yesterday's tool results are stale and resuming with them produces unreliable analysis. ✓

b. Resume using --resume — maintaining context continuity always outweighs data freshness.

c. Fork the prior session to create a clean branch for today's investigation.

d. Resume but immediately re-run all tool calls to refresh stale results.

Question 38

Flag question

Correct Mark 1.00 out of 1.00

Your manager proposes using the Batch API for both pre-merge checks and an overnight technical debt report. How do you evaluate this?

a. Batch both with a timeout fallback to real-time.

b. Switch both to Batch with status polling.

c. Batch for debt reports only; keep real-time API for pre-merge checks — the Batch API has up to 24-hour processing time, making it unsuitable for blocking workflows. ✓

d. Keep real-time API for both.

Question 39

Flag question

Correct Mark 1.00 out of 1.00

A developer wants to experiment with a different code formatting style for their local workflow. How should they configure this without affecting any other team members?

a. Configure the preferences in ~/.claude/CLAUDE.md — user-level configuration is stored locally on each developer's machine, is never committed to version control, and does not affect teammates. ✓

b. Create a local-preferences.md file in the project .claude/ directory.

c. Create a personal formatting branch with a modified project CLAUDE.md.

d. Add their preferences to a personal section of the project CLAUDE.md.

Question 40

Flag question

Correct Mark 1.00 out of 1.00

Your customer support agent compresses conversation history every 10 turns. A customer states their premium membership status in turn 2. By turn 35, the summary says 'customer has an account.' A discount the customer was entitled to is never applied. What is the fix?

a. Ask customers to re-state their membership status at the start of every session.
b. Extend the compression interval to 20 turns.
c. Extract account attributes — membership tier, credit limit, active promotions, account flags — into an immutable structured facts block maintained outside the compressible conversation history. ✓
d. Instruct the summariser: 'Always include membership tier in summaries.'

Question 41

Flag question

Correct Mark 1.00 out of 1.00

Your extraction pipeline reports field-level confidence scores but cannot route documents to human review fast enough — reviewers are overwhelmed with 2,000 documents per day. You need to prioritize which documents need human review most urgently. What is the correct approach?

a. Route only documents where the model explicitly flags an error.
b. Route documents in order of submission — first in, first reviewed.
c. Route based on compound risk: prioritize documents where low-confidence fields are also high-stakes (e.g., total_amount, legal_terms) over documents where only low-stakes fields (e.g., document_reference_number) are uncertain — this focuses reviewer capacity on highest-risk extractions. ✓
d. Route all documents with any field confidence below 0.99 to review.

Question 42

Flag question

Correct Mark 1.00 out of 1.00

Your extraction pipeline processes invoices where stated_total sometimes doesn't match the sum of line items. You want to automatically detect these discrepancies in the extraction step. What is the correct design?

a. Post-process the extracted JSON and compute the difference between total fields after extraction.
b. Use a separate validation API call to compare total fields after the main extraction.
c. Add a conflict_detected boolean field to the output schema and include a prompt instruction to set it to true when calculated_total and stated_total differ — this surfaces the discrepancy directly in the structured output. ✓
d. Route all invoices with totals to human review regardless of whether they match.

Question 43

Flag question

Correct Mark 1.00 out of 1.00

Your document extraction pipeline extracts a calculated_total by summing line items and compares it to the stated_total from the document. 3% of documents show discrepancies. You want to route only these discrepant documents for human review without reviewing all 97%. What is the correct design?

a. Route all documents where stated_total > $10,000 to human review as a risk-based proxy.
b. Flag all documents where stated_total ends in a round number as potentially suspicious.
c. Have the model automatically correct the stated_total to match the calculated_total before outputting.
d. Include both calculated_total and stated_total in the output schema with a conflict_detected boolean; in the pipeline, filter for conflict_detected == true and route only those documents to the human review queue. ✓

Question 44

Flag question

Correct Mark 1.00 out of 1.00

A customer contacts support with three separate issues in one session. After several turns, the agent begins confusing which issue it is currently addressing. What is the most appropriate architectural fix?

a. Increase the model's context window so the full conversation fits without compression.
b. Structure the system prompt with a persistent 'current issue tracker' — a live list of open issues and their statuses — that the agent updates after each topic transition. ✓
c. Spawn a separate agent for each issue after the first one is identified.
d. Limit sessions to one issue at a time and ask the customer to call back for each additional issue.

Question 45

Flag question

Correct Mark 1.00 out of 1.00

You need guaranteed structured output from Claude but your pipeline encounters three different document types — invoice, purchase order, receipt — each with a different extraction schema. You do not know the document type before sending to Claude. Which tool_choice approach correctly handles this?

a. Define all three extraction tools and set tool_choice: {type: 'any'} — this forces Claude to call one tool (guaranteeing structured output) while allowing it to select the most appropriate schema based on the document's content. ✓
b. Run three separate API calls and use the result with the highest confidence score.
c. Use tool_choice: auto and list all three schema tools.
d. Force tool_choice: {type: 'tool', name: 'invoice_extractor'} and default to the most common schema.

Question 46

Flag question

Correct Mark 1.00 out of 1.00

Your review prompt says 'report issues accurately and consistently.' The agent produces 40% false positives on style suggestions, which developers are dismissing and losing trust in the system. How do you fix this?

a. Instruct the agent to 'be conservative' and prefer under-reporting.
b. Tell the agent to 'only report high-confidence findings.'
c. Write explicit categorical criteria defining which issues to report (e.g., 'flag only when comment behavior contradicts actual code') and temporarily disable the high-false-positive category while improving its prompt. ✓
d. Add a confidence threshold field to the output schema and filter low-confidence findings client-side.

Question 47

Flag question

Correct Mark 1.00 out of 1.00

Your pipeline processes 1,000 invoices per hour. A critical validation service used for 15% of invoices goes down for 2 hours. What is the correct operational pattern to maintain pipeline throughput during the outage?

a. Process all invoices without validation during the outage and re-validate everything afterward.
b. Continue processing the 85% of invoices that do not require the validation service; for the 15% that do, queue them with a 'validation_pending' status and process them when the service recovers — structured partial processing is better than full stoppage. ✓
c. Stop the entire pipeline until the validation service recovers.
d. Switch to the synchronous API for the 15% requiring validation to bypass the service.

Question 48

Flag question

Correct Mark 1.00 out of 1.00

Your contract extraction schema marks governing_law as required. For 25% of contracts that omit it, the model fabricates legal jurisdictions. What schema change prevents fabrication?

a. Change governing_law to optional and nullable (type: ['string', 'null']) — the model can return null when the clause is genuinely absent instead of fabricating content to satisfy a required field. ✓
b. Add an enum of all valid legal jurisdictions the model can choose from.
c. Add a validation step that rejects extractions with missing governing_law.
d. Remove governing_law from the schema since it is absent too frequently to be reliable.

Question 49

Flag question

Correct Mark 1.00 out of 1.00

You need to process 80,000 documents over a weekend compliance audit. Results needed by Monday morning. Current synchronous API cost would be $1,200. What is the correct API strategy?

a. Split: synchronous for complex documents, Batch API for simple ones.
b. Use the Message Batches API — the 60+ hour window comfortably fits the 24-hour batch processing guarantee and the 50% cost reduction saves $600 with no workflow impact. ✓
c. Use the synchronous API for a safer deadline margin.
d. Use the synchronous API with 30 parallel workers to maximize throughput.

Question 50

Flag question

Correct Mark 1.00 out of 1.00

Your agent's conversation history now includes the results of 25 tool calls across a long session. The model is starting to miss details from the beginning of the conversation. Placing key information in what part of the context helps mitigate the 'lost in the middle' effect?

a. Distributed evenly across the full conversation history to maximize recall.
b. In the middle of the conversation where the model has already processed the prior context.
c. At the end of the most recent user turn, since the model prioritizes recent input.
d. Near the beginning of the context (system prompt or top of the prompt) and near the end of the current input — models reliably process information at the start and end of long inputs. ✓

Question 51

Flag question

Correct Mark 1.00 out of 1.00

Contract dates appear as '03/12/2024', '3rd December 2024', 'late 2024', and 'Q4 2024'. A regex post-processor handles the first two but fails on the last two. What extraction approach handles all four formats?

a. Reject contracts with informal date formats.
b. Instruct the model to interpret and normalise all date expressions to ISO 8601 during extraction with few-shot examples for each format — the model's semantic understanding handles 'late 2024' and 'Q4 2024' that regex cannot reliably parse. ✓
c. Create a separate normalisation API call for each date format type.
d. Extract only dates matching standard numeric formats.

Question 52

Flag question

Correct Mark 1.00 out of 1.00

Your customer support agent compresses conversation history every 10 turns. A customer states their premium membership status in turn 2. By turn 35, the summary says 'customer has an account.' A discount the customer was entitled to is never applied. What is the fix?

a. Extend the compression interval to 20 turns.
b. Extract account attributes — membership tier, credit limit, active promotions, account flags — into an immutable structured facts block maintained outside the compressible conversation history. ✓
c. Instruct the summariser: 'Always include membership tier in summaries.'
d. Ask customers to re-state their membership status at the start of every session.

Question 53

Flag question

Correct Mark 1.00 out of 1.00

Your agent processes customer order data where each lookup returns 50 fields across 12 average orders per session. By turn 30, the agent starts referencing outdated order statuses from turn 5. What causes this and what prevents it?

a. The model has a memory bug that prioritises earlier responses.
b. Tool results appended verbatim across 30+ turns accumulate redundant data and crowd out recent information. Trim each order lookup response to only the fields relevant to the current task before appending to context, and replace superseded status data rather than appending. ✓
c. Increase max_tokens so the model can hold all 30 turns of order data.
d. Switch to a model with larger context window to handle extensive order histories.

Question 54

Flag question

Correct Mark 1.00 out of 1.00

Your agent's conversation history now includes the results of 25 tool calls across a long session. The model is starting to miss details from the beginning of the conversation. Placing key information in what part of the context helps mitigate the 'lost in the middle' effect?

a. In the middle of the conversation where the model has already processed the prior context.
b. Near the beginning of the context (system prompt or top of the prompt) and near the end of the current input — models reliably process information at the start and end of long inputs. ✓
c. Distributed evenly across the full conversation history to maximize recall.
d. At the end of the most recent user turn, since the model prioritizes recent input.

Question 55

Flag question

Correct Mark 1.00 out of 1.00

Your order management agent accumulates tool results from 20+ order lookups across a long session. By turn 50, it references an order status from turn 8 that was later superseded by a status update in turn 35. What context management fix prevents this?

a. Structure the agent to replace or annotate superseded tool results rather than indefinitely appending all results — when a newer lookup for the same order arrives, update or replace the prior result in the structured context rather than leaving both. ✓

b. Increase max_tokens so all 50 turns of order data can be held simultaneously.

c. Add a system prompt instruction: 'Always use the most recent tool result for each order.'

d. Use /compact every 15 turns to remove outdated order statuses.

Question 56

Flag question

Correct Mark 1.00 out of 1.00

Your support agent compresses conversation history every 8 turns. A customer states their corporate account discount tier (15%) in turn 3. By turn 40, the summary says 'customer has discount.' The agent applies the wrong discount percentage. What prevents this?

a. Ask customers to confirm their discount tier at the start of every session.

b. Add 'preserve exact discount percentages' to the summariser instructions.

c. Extract structured account facts — discount tier, percentage, eligible products — into an immutable facts block outside the compressed conversation history so exact values survive compression intact. ✓

d. Extend the compression cycle to 15 turns.

Question 57

Flag question

Correct Mark 1.00 out of 1.00

Your agent summarizes conversation history every 10 turns. A customer states their refund amount as $347.50 in turn 3. By turn 25, the summary reads 'customer requested a refund for a moderate amount.' What context management technique preserves the exact value?

a. Increase the summarization interval from 10 turns to 20 turns.

b. Extract transactional values — amounts, order IDs, dates, account numbers — into a persistent 'case facts' block maintained outside the summarized history, so exact figures are never exposed to summarization loss. ✓

c. Instruct the summarizer: 'Always preserve exact dollar amounts in summaries.'

d. Use higher precision summarization by increasing the summary's max_tokens.

Question 58

Flag question

Correct Mark 1.00 out of 1.00

Your agentic loop appends every tool result verbatim to the conversation history. The agent uses an order lookup tool that returns 47 fields per call. After 12 lookups across a long session, the agent starts missing details stated earlier. What is the most likely cause and fix?

a. Switch from sequential to parallel tool calls to reduce the total number of lookups.
b. Increase max_tokens to give the model more room to hold all tool results in context.
c. The model has a bug causing it to ignore earlier messages — upgrade to a newer model version.
d. Tool results accumulate in context and consume tokens disproportionately to their relevance. Trim each tool response to only the fields relevant to the current task before appending it to history. ✓

Question 59

Flag question

Correct Mark 1.00 out of 1.00

After a 20-turn conversation handling multiple orders, the agent references the wrong order number when processing a final refund. Order numbers were stated correctly in early turns. What context management technique prevents this?

a. Extract critical transactional facts — customer ID, order numbers, amounts, statuses — into an immutable 'case facts' block at the top of every prompt, kept separate from summarized conversational history. ✓
b. Summarize the conversation every 5 turns to compress history and keep context manageable.
c. Increase max_tokens so the model can hold the full conversation in its active context window.
d. Add a system prompt instruction: 'Before processing any refund, re-read the full conversation to confirm the order number.'

Question 60

Flag question

Correct Mark 1.00 out of 1.00

Your agent processes customer account data where each lookup returns 55 fields for each of 10 average accounts per session. By turn 25, the agent references outdated account statuses from early turns. What is the root cause and fix?

a. Use /compact every 10 turns to remove outdated account statuses.
b. Verbose tool results accumulate across turns, consuming context disproportionately and pushing important recent information out of effective attention range. Trim each account lookup to only the fields relevant to the current task and replace superseded statuses rather than appending indefinitely. ✓
c. The model has a caching bug that prioritises earlier responses.
d. Increase max_tokens to accommodate all account data across 25 turns.

Previous activity
Certified Architect: Coach Connect 16

Jump to...